

Manual

Overview

This project is a configuration performance tuning tool for discrete benchmark datasets. It compares two search strategies: Baseline Random Search and an improved BestConfig style search. The tool reads configuration-performance datasets from `datasets/`, searches for strong configurations under a fixed budget, and writes outputs for analysis and reporting.

Inputs

The tool expects CSV files stored in datasets/.

For each dataset:

1. Every column except the last is treated as a configuration parameter
2. The last column is treated as the performance metric
3. The filename stem is used as the system name

The repository includes the benchmark datasets:

1. 7z
2. Apache
3. brotli
4. LLVM
5. PostgreSQL
6. spear
7. storm
8. x264

Main Scripts

1. Baseline run

```
python -m scripts.run_baseline
```

Runs the baseline random search once across all datasets.

Settings:

- Budget: 100 valid measurements per dataset
- Seed: 42

Outputs are written to results/baseline_random_search/.

2. Improved run

```
python -m scripts.run_improved
```

Runs the improved BestConfig-style search once across all datasets.

Settings:

- Budget: 100 valid measurements per dataset
- Seed: 42

Outputs are written to results/improved_bestconfig/.

3. Repeated experiments

```
python -m scripts.run_experiments
```

Runs the repeated comparison experiment across both algorithms.

Settings:

- Algorithms: baseline_random_search and bestconfig
- Budgets: 50 and 100
- Runs per setting: 30
- Base seed: 42

Outputs are written to results/experiments/.

4. Comparison plots

```
python -m scripts.make_comparison_plots
```

Creates comparison plots from the experiment results.

Outputs:

- results/experiments/plots/boxplots/
- results/experiments/plots/median_comparisons/

5. Statistical tests

```
python -m scripts.run_stat_tests
```

Runs paired Wilcoxon signed-rank tests and applies Holm correction.

Output:

- results/experiments/stat_tests/wilcoxon_results.csv

6. Final report table

```
python -m scripts.build_results_table
```

Builds the final compact results table and writes it to

`results/experiments/report\_tables/final\_results\_table.csv` and

`results/experiments/report\_tables/final\_results\_table.md`.

How to Use the Tool

Use the scripts depending on what you want to do:

1. Run `scripts.run_baseline` if you only want the baseline results
2. Run `scripts.run_improved` if you only want the improved-method results
3. Run `scripts.run_experiments` if you want repeated comparisons across datasets and budgets
4. Run the plotting, statistical testing, and table-building scripts after experiments if you want analysis outputs

Troubleshooting

Missing package errors

Install dependencies again with:

```
python -m pip install -r requirements.txt
```

Missing input files

Check that the repository still contains:

datasets/*.csv for search runs

The required files in results/experiments/ before running downstream scripts

High `invalid_count` values

This is not necessarily an error. The search methods may sample configurations that are not present in a dataset. In that case, the code records an invalid sample, assigns a worst case performance value, and does not count it toward the valid measurement budget.